

MANUAL DE USO

Llama 3.2 3B Instruct via HuggingFace

Guia de configuração, autenticação e execução do ambiente

1. Visão Geral

Este manual descreve o passo a passo para configurar o ambiente e executar o case utilizando o modelo de linguagem Llama 3.2 3B Instruct, disponibilizado pela Meta via plataforma HuggingFace.

Componente	Detalhe
Modelo	Llama 3.2 3B Instruct (meta-llama)
Plataforma	HuggingFace (huggingface.co)
Requisito de Hardware	GPU com mínimo 8 GB de VRAM (recomendado 15GB de VRAM)
Notebook principal	main.ipynb
Autenticação	HuggingFace Access Token (CLI)

2. Pré-Requisitos

Antes de iniciar, certifique-se de que os itens a seguir estão disponíveis:

- Conta ativa no HuggingFace (huggingface.co)
- Acesso aprovado ao modelo [meta-llama/Llama-3.2-3B-Instruct · Hugging Face](#)
- GPU com pelo menos 8 GB de VRAM disponíveis
- Ambiente Python configurado com ambiente virtual
- Acesso ao repositório do projeto

 **Atenção:** O acesso ao modelo Llama 3.2 deve ser solicitado diretamente na página do modelo no HuggingFace. A aprovação pode levar alguns minutos. Além disso, redes corporativas podem bloquear o acesso ao site — utilize uma conexão alternativa se necessário.

3. Obtendo o Código

Clone o repositório do projeto em seu ambiente local ou em sua máquina de desenvolvimento. O link do repositório: [gu1m/data_master_at_data_master](https://github.com/gu1m/data_master_at_data_master).

Após clonar, a estrutura de arquivos esperada é:

```
? main.ipynb
? Sistema_Especialista_GRC_COSO_ERM_2017.docx.pdf
?
?
+---llama
? ? llama_logs.jsonl
? ?
+---memory_support
? ? memory_cleaner.py
? ? memory_purifier.py
? ? memory_support.py
? ?
? ?
? ?
+---memory_support_db
? ? chroma.sqlite3
? ?
? ? +---f0237d3f-40dd-4804-ae99-b71168363bb7
? ?     data_level0.bin
? ?     header.bin
? ?     length.bin
? ?     link_lists.bin
? ?
+---RAG
? ? BuildVectorStore.py
? ? generate_db.ipynb
? ? rank_context.py
? ? __init__.py
? ?
? ?
? ?
? ? +---db
? ? ? chroma.sqlite3
? ? ?
? ? ? +---a27c328c-86b6-4539-bf63-ce32f037a9d3
? ? ?     data_level0.bin
? ? ?     header.bin
? ? ?     length.bin
? ? ?     link_lists.bin
? ? ?
? ? +---docs_uploader
? ? +---documents_txt
? ? ? ? 854173331-Coso-Erm-Standard-2017.txt
? ? ? ?
? ? ? ?
? ? +---old
? ?     -ERM-COSO-Application-Techniques.txt
? ?     a6ae4494-56a8-4e76-9fe1-57d04438b4a3.pdf
? ?     cobit5.pdf
? ?     DORA.pdf
? ?     DRO_pos_bacen.pdf
? ?     DRO_pre_bacen.pdf
? ?     itil-v4.pdf
? ?     NIST.CSWP.29.por.pdf
? ?
```

4. Solicitando Acesso ao Modelo

O modelo Llama 3.2 3B Instruct é disponibilizado pela Meta no HuggingFace com acesso controlado. Siga os passos abaixo para solicitar permissão de uso:

- Acesse a página do modelo em: [meta-llama/Llama-3.2-3B-Instruct · Hugging Face](https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct)
- Faça login na sua conta HuggingFace
- Clique em "Access repository" e aceite os termos de uso da Meta
- Aguarde a confirmação de acesso (geralmente imediata, mas pode levar minutos)

5. Gerando um Access Token

O Access Token é necessário para autenticar o download do modelo. Acesse as configurações de tokens em: huggingface.co/settings/tokens

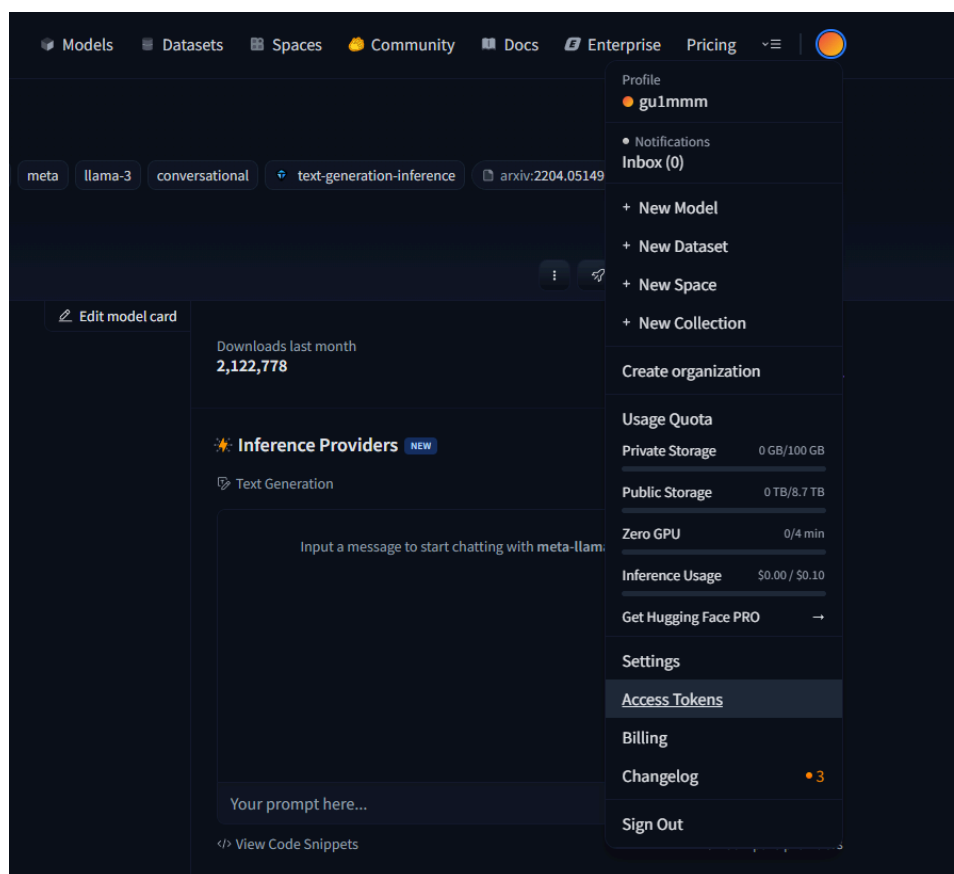


Figura 1 — Tela de Access Tokens no HuggingFace

Na tela de tokens, clique em "Create new Token" conforme ilustrado abaixo:

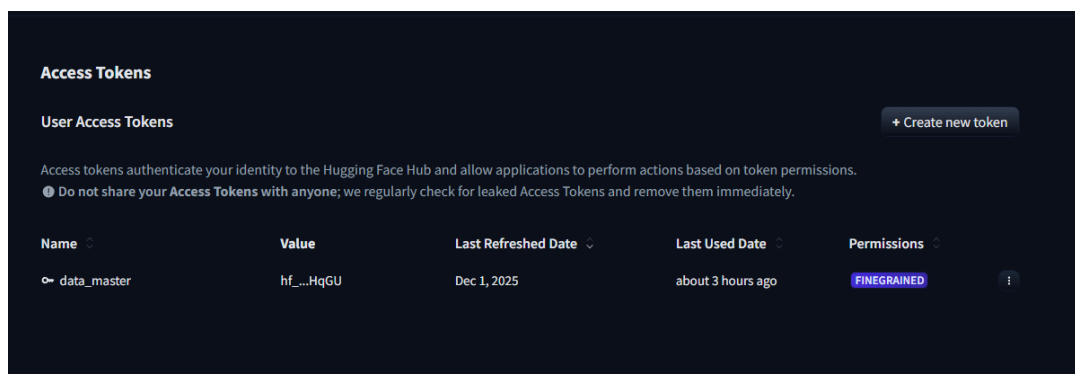


Figura 2 — Botão para criação de novo token

Na tela de criação do token:

- Insira um nome descritivo para o token (ex: llama-case)
- Marque todas as caixas em "User Permissions"
- Clique em "Create Token" ao final da página
- Copie e salve o token gerado em local seguro — ele não será exibido novamente

The screenshot shows the 'Create new Access Token' form. It has a 'Token type' section with 'Fine-grained' selected. Below it is a 'Token name' input field. The 'User permissions (gu1mmm)' section contains several categories of permissions, each with a list of checkboxes: 'Repositories' (4 items), 'Webhooks' (2 items), 'Discussions & Posts' (3 items), 'Inference' (3 items), 'Collections' (2 items), 'Billing' (1 item), and 'Jobs' (1 item). All checkboxes are currently checked.

Figura 3 — Formulário de criação de token com permissões de usuário



Atenção: Guarde o token com segurança. Trate-o como uma senha: não compartilhe publicamente nem inclua em repositórios de código.

6. Configurando o Ambiente

Com o token em mãos, configure o ambiente de execução seguindo a ordem abaixo:

#	Etapa	Instrução
1	Selecionar GPU	Escolha um ambiente com GPU de ao menos 8 GB de VRAM (ex: Google Colab Pro, Kaggle, servidor próprio).
2	Criar env	Crie um ambiente virtual para conseguir instalar as dependências sem ter conflitos com dependências já instaladas na sua máquina.
3	Instalar dependências	Abra o main.ipynb e execute a primeira célula, que contém os comandos de instalação das bibliotecas.
4	Abrir terminal	No ambiente, abra o terminal com o env ativo/selecionado.
5	Login HuggingFace	Execute o comando de autenticação com seu token.
6	Verificar login	Confirme que a autenticação foi bem-sucedida.

7. Autenticando no HuggingFace via Terminal

Com o terminal aberto e o .env ativo, execute o comando de login substituindo o token pelo valor gerado anteriormente:

```
huggingface-cli login --token hf_XXXXXXXXXXXXXXXXXXXX
```

Após pressionar Enter, verifique se a autenticação foi realizada com sucesso:

```
huggingface-cli whoami
```

Resultado	Significado
-----------	-------------

Exibe seu username	✓ Autenticado com sucesso — pronto para usar
Erro ou "Not logged in"	✗ Token inválido ou login não concluído

8. Executando o Notebook

Com o ambiente autenticado e as dependências instaladas, abra o arquivo `main.ipynb` e execute as células em sequência. Certifique-se de que a GPU está selecionada como acelerador de hardware no ambiente antes de executar.

Para confirmar no terminal rode:

```
nvidia-smi
```

Se aparecer algo muito similar como isso:

```

+-----+
| NVIDIA-SMI 566.14                | Driver Version: 566.14      | CUDA Version: 12.7          |
+-----+-----+
| GPU  Name                   | Driver-Model | Bus-Id  | Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf             | Pwr:Usage/Cap|          |         | GPU-Util  Compute M. |
|=====+=====+
| 0   NVIDIA GeForce RTX 4060  | WDDM        | 00000000:05:00.0  On      | N/A              |
| 34%   42C    P2              | N/A / 115W   | 7592MiB / 8188MiB | 100%    Default     |
|=====+=====+
|                               |              |          |         | MIG M.         |
|                               |              |          |         | N/A              |
+-----+-----+

```

O seu ambiente está com uma GPU dedicada, na 2ª coluna na 2ª linha é possível ver quanto de VRAM sua GPU tem disponível. No caso da imagem acima tem 8188MB VRAM (8gb VRAM).

```
[ ]: !pip install -U langchain chroma
!pip install tqdm --upgrade
!pip install --upgrade ipywidgets
!pip install -U langchain-huggingface
!pip install -q langchain_community
!pip install -q sentence-transformers
!pip install -q chromadb
!pip install pandas
!pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124
!pip install "unstructured[all-docs]"

[2]: from llama.call_llama import CallLlama
import warnings
warnings.filterwarnings("ignore")

[4]: llm = CallLlama()

Iniciando call_llama...
Iniciando sistema de memória: 20% | 1/5INFO:sentence_transformers.SentenceTransformer:Load pretrained SentenceTransformer: intfloat/multilingual-e5-smal
1
INFO:chromadb.telemetry.product.posthog:Anonymized telemetry enabled. See https://docs.trychroma.com/telemetry for more information.
Iniciando sistema de RAG: 40% | 2/5INFO:RAG: # Carregando embeddings...
INFO:sentence_transformers.SentenceTransformer:Load pretrained SentenceTransformer: BAAI/bge-m3
INFO:chromadb.telemetry.product.posthog:Anonymized telemetry enabled. See https://docs.trychroma.com/telemetry for more information.
Carregando modelo llama local: 60% | 3/5'torch.dtype' is deprecated! Use 'dtype' instead!
Loading widget...
Iniciando Answer Evaluation: 100% | 5/5
call_llama inicializado com sucesso!

[5]: resposta = llm.invoke(
    user = "Quanto e quais são componentes que estruturam o framework (EBM)?"
)

Recuperando memórias relevantes para: 'Quanto e quais são componentes que estruturam o ...'***

[11]: print(resposta)

O Framework de Gestão de Risco (GRC) é composto por 5 componentes principais, que são: ***

[ ]:
```

main.ipynb

Ao executar a célula de *invoke* do modelo, é esperado que uma quantidade significativa de informações seja exibida no output. Esse comportamento faz parte do processo de transparência e observabilidade do pipeline.

Inicialmente, será apresentada a busca por memórias semanticamente similares à pergunta realizada pelo usuário. Em seguida, serão exibidos os documentos recuperados pelo pipeline de RAG considerados relevantes para a construção da resposta.

Na sequência, será mostrado o fluxo de tratamento da memória, juntamente com métricas associadas à geração da resposta, incluindo os *logprobs* de cada token e a perplexidade.

Por fim, na célula `print(resposta)`, será possível visualizar a resposta final gerada pelo modelo.

evaluteRAG.ipynb

Para reperformar os testes de Hit Rate, MMR e Precision do pipeline de RAG no notebook **evaluteRAG.ipynb** isso é possível, se já foi instalado as dependências que estão nesse notebook, não é necessário executar a primeira célula dos pip's install. Após isso será possível ver a variável dataframe com [Query + Respectivo Chunk], após executar essa célula do dataframe, será possível testar com o próximo trecho de código.

```
]# 1. Constrói o pipeline (sua função já está correta com o caminho ../RAG/db)
pipeline = build_pipeline()

# 2. Instancia o Evaluator
evaluator = RAGEvaluator(
    pipeline_fn=pipeline,
    dataset=dataset,
    model_name="BAAI/bge-m3"
)

# 3. Roda a avaliação
# k=20 é o que o retriever (Chroma) busca antes do Reranker filtrar para top_n=6
# similarity_threshold=0.70
df = evaluator.evaluate(k=20, similarity_threshold=0.75)
```

Quando for concluída, podemos rodar o trecho de código que gera os gráficos com os resultados.


```

import matplotlib.pyplot as plt

# 1. Criamos a figura com 3 subplots verticais
fig, axs = plt.subplots(3, 1, figsize=(10, 15), sharex=True)

# 2. Atualizamos a lista de métricas para os novos nomes das colunas
# (coluna_no_df, nome_exibição, marcador, cor, offset_texto)
metrics = [
    ('hit_rate', 'Hit Rate (Success @ K)', 'o', 'blue', (0, 7)),
    ('mrr', 'MRR (Mean Reciprocal Rank)', 'D', 'purple', (0, 7)),
    ('precision_at_1', 'Precision @ 1', 's', 'darkred', (0, 7))
]

for ax, (col, label, marker, color, offset) in zip(axs, metrics):
    # Plotagem da linha
    ax.plot(df['similarity_threshold'], df[col],
            marker=marker, label=label, color=color, linewidth=2)

    # Adicionando anotações de dados (os valores numéricos sobre os pontos)
    for i in range(len(df)):
        val = df[col].iloc[i]
        threshold = df['similarity_threshold'].iloc[i]
        ax.annotate(f'{val:.2f}',
                    (threshold, val),
                    textcoords="offset points",
                    xytext=offset,
                    ha='center',
                    fontsize=9,
                    fontweight='bold')

    # Configurações estéticas de cada subplot
    ax.set_ylabel('Score (0 a 1)')
    ax.set_title(f'{label} vs. Similarity Threshold', fontsize=12, pad=10)
    ax.grid(True, linestyle='--', alpha=0.6)
    ax.set_ylim(-0.05, 1.05) # Garante que a escala seja sempre de 0 a 1
    ax.legend(loc='upper right')

# Eixo X comum
axs[-1].set_xlabel('Similarity Threshold (Corte de Similaridade)')

# Ajuste fino para evitar sobreposição
plt.tight_layout()
plt.show()

```

Esses gráficos são explicados no documento técnico.

```
[ ]: pip install langchain-core langchain-community langchain-text-splitters langchain-huggingface langchain-chroma

[ ]: pip install "unstructured[all-docs]"

[ ]: pip install regex

[ ]: pip install numpy==1.26.4

[ ]: pip show torch

[3]: from BuildVectorStore import PDFVectorStore

[4]: builder = PDFVectorStore()

INFO:RAG: Carregando embeddings...
INFO:sentence_transformers.SentenceTransformer:Load pretrained SentenceTransformer: BAAI/bge-m3

[5]: builder._build_db()

INFO:RAG: documentos carregados: 1
INFO:RAG: seções detectadas: 96
INFO:RAG: chunks gerados: 301
INFO:RAG: hash removidos: 0
INFO:RAG: semantic removidos: 8
INFO:RAG: Gerando embeddings + salvando Chroma...
INFO:chromadb.telemetry.product.posthog:Anonymized telemetry enabled. See https://docs.trychroma.com
INFO:RAG: DB persistido!

[5]: <langchain_chroma.vectorstores.Chroma at 0x2ce7f400c20>

[ ]:
```

generate_db.ipynb

Nesse notebook é realizado a criação do vector store utilizado no RAG, se for reperformar essa funcionalidade é necessário excluir os arquivos que estão no caminho:

(data_master/llama/RAG/db)

Exclua todos os arquivos dentro da pasta db. Após isso pode rodar as células de pip install se necessário e as subseqüentes para realizar geração do vector store.

9. Resolução de Problemas Comuns

Problema	Solução
Rede corporativa bloqueando download	Use uma rede doméstica ou VPN pessoal para baixar o modelo.
Token inválido ou expirado	Gere um novo token no HuggingFace e repita o comando de login.
CUDA out of memory	Certifique-se de que a GPU tem ao menos 8 GB de VRAM livres. Feche outros processos que usem GPU.
Acesso negado ao modelo	Confirme que sua solicitação de acesso ao modelo foi aprovada na página do HuggingFace.

Dependências faltando

Execute novamente a primeira célula do main.ipynb para instalar todas as bibliotecas.